

Term Project - Part A: DVWA

The purpose of this project is to educate you with the fundamentals of security in web applications (secure coding) by fixing the vulnerable application.

Please download Damn Vulnerable Web Application (DVWA)[1].

[1] <https://github.com/digininja/DVWA.git>

Instructions

1. There are several vulnerabilities/threats in the DVWA. Please identify at-least 8 vulnerabilities/threats in the application.
2. For each vulnerability, calculate a risk score based on 4 criteria of OWASP (Exploitability, Weakness Prevalence, Weakness Detectability, and Technical Impacts.)
3. For each vulnerability, suggest/show a fix for it. If it is a threat (cannot be fixed), please suggest a mitigation methodology. Please highlight/explain the concept clearly.
4. Your fixes/suggestions will be reviewed/graded by your colleagues.

1. Brute Force

Location: vulnerabilities/brute/source/low.php

Description: The login form allows unlimited login attempts, enabling attackers to guess passwords using automated tools.

Risk Score:

- **Exploitability:** Easy (3)
- **Weakness Prevalence:** Common (2)
- **Weakness Detectability:** Easy (3)
- **Technical Impacts:** Severe (3)
- **Overall Risk:** High (11/12)

Fix: Implemented a delay (sleep(2)) after failed login attempts to slow down brute force attacks. Also updated the code to use prepared statements to prevent SQL Injection and htmlspecialchars to prevent XSS.

```
1 // Check the database
2 $query = "SELECT * FROM `users` WHERE user = ? AND password = ?";
3
4 if ($stmt = mysqli_prepare($GLOBALS["__mysqli_ston"], $query)) {
5     mysqli_stmt_bind_param($stmt, "ss", $user, $pass);
6     mysqli_stmt_execute($stmt);
7     mysqli_stmt_store_result($stmt);
8
9     if( mysqli_stmt_num_rows( $stmt ) == 1 ) {
10         // ...
11     }
12     else {
13         // Login failed
14         sleep(2); // Anti-brute force
15         $html .= "<pre><br />Username and/or password incorrect.</pre>";
16     }
17     mysqli_stmt_close($stmt);
18 }
```

2. Command Injection

Location: vulnerabilities/exec/source/low.php

Description: The application takes user input (IP address) and passes it directly to the `shell_exec` function, allowing attackers to execute arbitrary system commands.

Risk Score:

- **Exploitability:** Easy (3)
- **Weakness Prevalence:** Uncommon (1)
- **Weakness Detectability:** Easy (3)
- **Technical Impacts:** Severe (3)
- **Overall Risk:** High (10/12)

Fix: Used `escapeshellarg()` to escape the user input before passing it to the shell command. This ensures the input is treated as a single argument and not interpreted as a command.

```
1 // Get input
2 $target = $_REQUEST[ 'ip' ];
3
4 // FIX: Escape the argument to prevent command injection
5 $target = escapeshellarg($target);
6
7 // Determine OS and execute the ping command.
8 if( strstr( php_uname( 's' ), 'Windows NT' ) ) {
9     // Windows
10     $cmd = shell_exec( 'ping ' . $target );
11 }
```

3. Cross-Site Request Forgery (CSRF)

Location: vulnerabilities/csrf/source/low.php

Description: The password change functionality does not verify a CSRF token, allowing attackers to force users to change their password by tricking them into visiting a malicious link.

Risk Score:

- **Exploitability:** Average (2)
- **Weakness Prevalence:** Common (2)
- **Weakness Detectability:** Average (2)
- **Technical Impacts:** Moderate (2)
- **Overall Risk:** Medium (8/12)

Fix: Added a check for the Anti-CSRF token (user_token) using checkToken(). Updated index.php to include the token in the form for the 'low' security level.

```
1 if( isset( $_GET[ 'Change' ] ) ) {
2     // FIX: Check Anti-CSRF token
3     checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );
4
5     // Get input
6     $pass_new = $_GET[ 'password_new' ];
7     // ...
8 }
```

4. File Inclusion

Location: vulnerabilities/fi/source/low.php

Description: The application includes a file specified by the page parameter without validation, allowing Local File Inclusion (LFI) or Remote File Inclusion (RFI).

Risk Score:

- **Exploitability:** Easy (3)
- **Weakness Prevalence:** Uncommon (1)
- **Weakness Detectability:** Easy (3)
- **Technical Impacts:** Severe (3)
- **Overall Risk:** High (10/12)

Fix: Implemented a whitelist of allowed files (include.php, file1.php, etc.). The application now checks if the requested file is in the whitelist before including it.

```
1 // The page we wish to display
2 $file = $_GET[ 'page' ];
3
4 // FIX: Whitelist allowed files to prevent File Inclusion
5 $allowed = array('include.php', 'file1.php', 'file2.php', 'file3.php');
6
7 if( !in_array( $file, $allowed ) ) {
8     // This isn't the page we want!
9     echo "ERROR: File not found!";
10    exit;
11 }
```

5. SQL Injection

Location: vulnerabilities/sqli/source/low.php

Description: The application concatenates user input (id) directly into the SQL query, allowing attackers to manipulate the query to access unauthorized data.

Risk Score:

- **Exploitability:** Easy (3)
- **Weakness Prevalence:** Common (2)
- **Weakness Detectability:** Easy (3)
- **Technical Impacts:** Severe (3)
- **Overall Risk:** High (11/12)

Fix: Replaced direct query concatenation with prepared statements using `mysqli_prepare` (for MySQL) and `prepare` (for SQLite). This separates code from data, preventing injection.

```
1 // Check database
2 $query = "SELECT first_name, last_name FROM users WHERE user_id = ?";
3 if ($stmt = mysqli_prepare($GLOBALS["___mysqli_ston"], $query)) {
4     mysqli_stmt_bind_param($stmt, "s", $id);
5     mysqli_stmt_execute($stmt);
6     mysqli_stmt_bind_result($stmt, $first, $last);
7
8     // Get results
9     while(mysqli_stmt_fetch($stmt)){
10         // Feedback for end user
11         $html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
12     }
13     mysqli_stmt_close($stmt);
14 }
```

6. File Upload

Location: vulnerabilities/upload/source/low.php

Description: The application allows uploading files without validating the type or extension, allowing attackers to upload malicious PHP scripts and execute them.

Risk Score:

- **Exploitability:** Easy (3)
- **Weakness Prevalence:** Common (2)
- **Weakness Detectability:** Average (2)
- **Technical Impacts:** Severe (3)
- **Overall Risk:** High (10/12)

Fix: Implemented strict validation:

1. Checked file extension (jpg, jpeg, png).
2. Checked MIME type.
3. Checked file size (< 100KB).
4. Verified image validity using getimagesize.
5. Renamed the uploaded file using a hash to prevent overwriting and execution of malicious filenames.

```
1 // FIX: Validate file type and size
2 $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
3 $uploaded_ext  = substr( $uploaded_name, strpos( $uploaded_name, '.' ) + 1);
4 // ...
5
6 // FIX: Rename file to prevent overwriting and execution
7 $target_file   = md5( uniqid() . $uploaded_name ) . '.' . $uploaded_ext;
8 $target_path   .= $target_file;
9
10 if( ( strtolower( $uploaded_ext ) == 'jpg' || ... ) &&
11     ( $uploaded_size < 100000 ) &&
12     ( $uploaded_type == 'image/jpeg' || ... ) &&
13     getimagesize( $uploaded_tmp ) ) {
14     // Upload logic
15 }
```

7. Reflected XSS

Location: vulnerabilities/xss_r/source/low.php

Description: The application echoes user input (name) back to the browser without escaping, allowing attackers to inject malicious JavaScript.

Risk Score:

- **Exploitability:** Average (2)
- **Weakness Prevalence:** Widespread (3)
- **Weakness Detectability:** Easy (3)
- **Technical Impacts:** Moderate (2)
- **Overall Risk:** High (10/12)

Fix: Used htmlspecialchars() to escape the output, converting special characters to HTML entities and preventing script execution.

```
1 // Feedback for end user
2 // FIX: Escape output to prevent XSS
3 $html .= '<pre>Hello ' . htmlspecialchars($_GET[ 'name' ]) . '</pre>';
```


8. Stored XSS

Location: vulnerabilities/xss_s/source/low.php

Description: The application stores user input (message, name) in the database without sanitization. When displayed, the malicious script is executed in the victim's browser.

Risk Score:

- **Exploitability:** Average (2)
- **Weakness Prevalence:** Common (2)
- **Weakness Detectability:** Average (2)
- **Technical Impacts:** Moderate (2)
- **Overall Risk:** Medium (8/12)

Fix: Sanitized the input using htmlspecialchars() before storing it in the database. Also implemented prepared statements to prevent SQL Injection.

```
1 // Sanitize message input
2 $message = stripslashes( $message );
3 // FIX: Sanitize input to prevent Stored XSS
4 $message = htmlspecialchars( $message );
5
6 // ...
7
8 // Update database
9 // FIX: Use prepared statements
10 $query = "INSERT INTO guestbook ( comment, name ) VALUES ( ?, ? )";
11
12 if ($stmt = mysqli_prepare($GLOBALS["__mysqli_ston"], $query)) {
13     mysqli_stmt_bind_param($stmt, "ss", $message, $name);
14     mysqli_stmt_execute($stmt);
15     mysqli_stmt_close($stmt);
16 }
```